Infrastructure as Intent (IAI)

Whitepaper II: SecOps Deep Dive

Second in a three-part whitepaper series on Infrastructure as Intent

Elaia Raj

2026

Abstract

Security has improved a great deal, but most of it still happens after infrastructure is built. Scanners check code that is already written. Reviewers look at changes that are already proposed. Posture tools find exposure that already exists in running systems. In other words, security today is mostly detection and fixing.

The first paper in this series introduced Infrastructure as Intent, or IAI: a reasoning layer above existing infrastructure tools, where a human states an outcome in plain language and an agent reads a declared source of truth, selects the right execution engines, generates the artefacts, validates the change for correctness, security, and cost, and seeks approval before anything is applied. Security was one of the three validation gates. This paper takes that gate and looks at it in depth.

The main argument is simple. IAI changes where security happens. When an agent generates the infrastructure, the organization's security rules can be applied while the change is written, not after it is written. The misconfiguration is not caught later. It is never created. Security stops being a separate stage and becomes part of how the infrastructure is produced.

This is not a new direction for the security field. The industry already agrees that detection alone is not enough, that the software supply chain is now a main source of risk, that policy belongs in code, and that AI agents in security operations should work under human approval. IAI brings these ideas together in one layer that covers cloud, on-premise virtualization, and the physical hardware beneath them.

1. Introduction and Thesis

Most security tooling is built on one assumption: insecure infrastructure will be created, so it must be found and fixed. Scanners exist to find misconfiguration after it is written. Reviewers exist to catch a change after it is proposed. Posture tools exist to discover exposure after it is already running. Each of these is a response to a problem that the system allowed to happen in the first place.

The thesis of this paper is that the assumption is no longer required. When an agent generates the infrastructure instead of a human, the organization's security rules can be made a condition of generation rather than a test applied afterwards. The agent does not write an insecure configuration and then submit it for scanning. It writes a configuration that follows policy, because policy decides what it is allowed to write. The misconfiguration is never created, so there is nothing to detect.

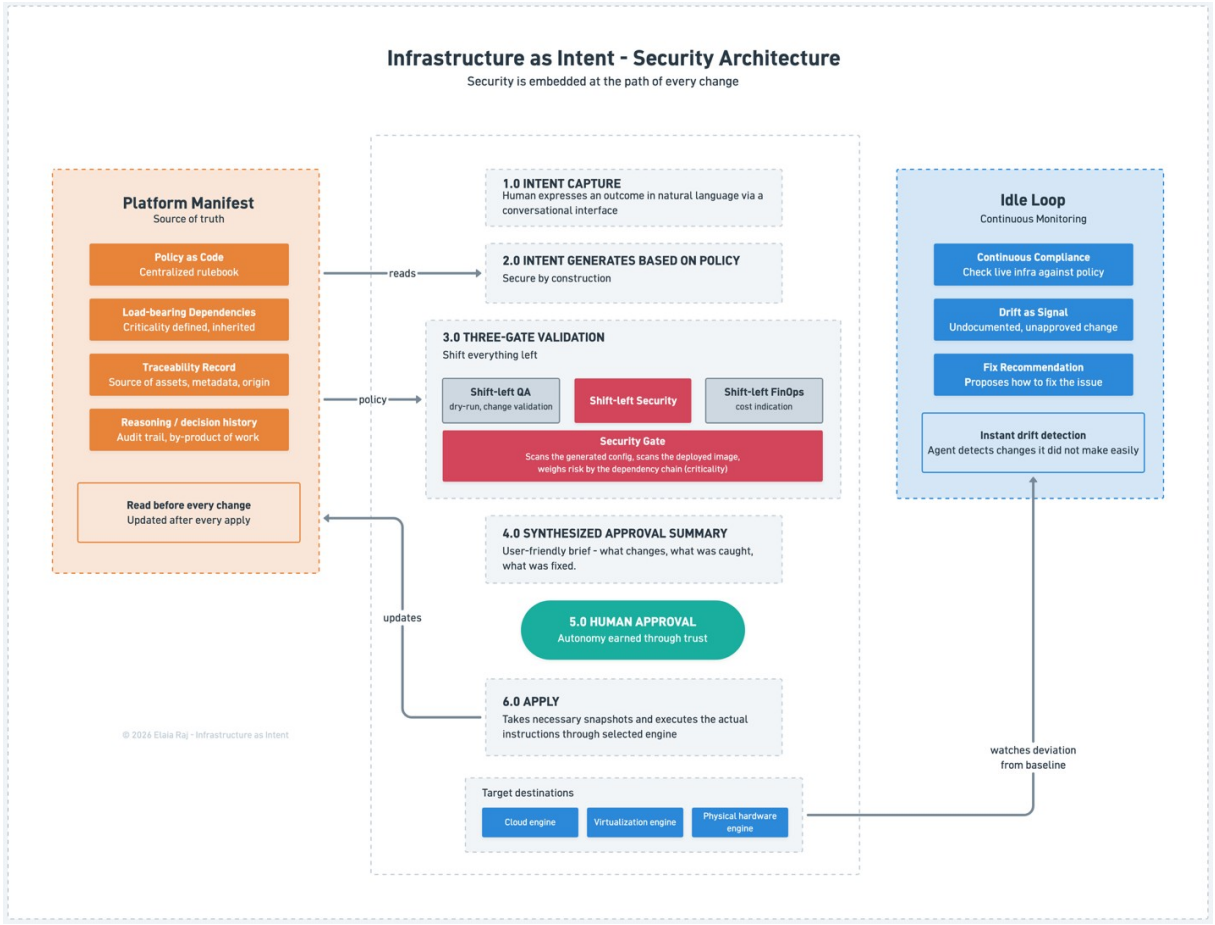

Figure 1. The end-to-end security architecture: the Platform Manifest holds policy and load-bearing context, the agent generates each change against policy and runs it through the three gates, a human approves a single summary, and the idle loop keeps watch after apply — one path every change passes through.

This is also why IAI makes security operations stronger, rather than just adding another tool. Every change goes through the intent layer, so a security check placed there applies to all of them. It is not a gate in a pipeline that can be manually overridden by an engineer — the agent is the one responsible for effecting the change, so the check cannot be skipped. And any change made outside that path does not stay hidden: the system detects it, as Section 7 explains.

2. Why Detection Is Not Enough

Detection on its own has reached its limit. Surveys of cloud environments published in 2026 report that most organizations; around seven in eight — are running at least one known, exploitable vulnerability in a live service [1]. This is not because the tools are weak. The tools are good, and there are many of them. The problem is the model they work in. Issues are found faster than they can be fixed, the backlog grows, and the gap between knowing and fixing gets wider, not smaller. A model

that generates against policy attacks this directly, because a problem that is never created produces no finding to triage.

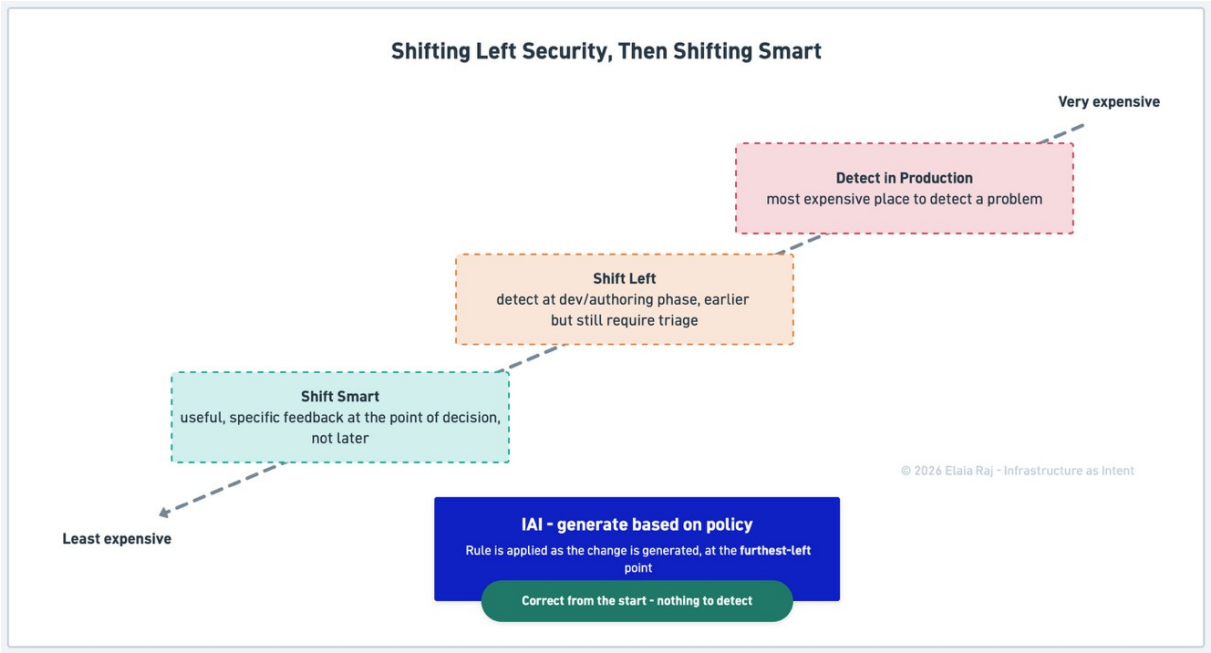

Figure 2. Security keeps moving earlier: from detecting problems in production, to shifting left at the authoring stage, to shifting smart with useful, specific feedback at the point of decision. IAI reaches that end state by applying the rule as the change is generated.

Shifting security left was the right move, and it still is. The next step is not to abandon it but to make it smarter — to shift *smart*: feedback that is useful and specific, given at the point of decision, rather than a larger pile of findings delivered earlier [2]. IAI takes the same idea to its furthest point. Rather than only moving the check earlier, it moves the rule into the moment the change is generated, so the change is produced correctly in the first place. At the same time, the main source of risk has moved. It is no longer only an organization's own configuration. It is what that configuration brings in — dependencies, base images, build systems, and third-party artefacts.

Securing the code you write is not enough when most of what runs is code you did not write. In 2025 alone, researchers found hundreds of thousands of new malicious open-source packages [3]. And AI agents are now entering security work, but the field is careful with them: in one 2026 survey of security leaders, only about one in seven would let an agent fix a problem with no human in the loop [4].

These points line up with IAI rather than against it:

- Detection alone is not enough.
- Shifting left worked; the next gain is making it smarter.
- What the infrastructure is built from matters as much as how it is configured.

- AI agents should act under human approval.

IAI does not argue with any of these. It takes all four and builds them into one layer that applies to the whole infrastructure at once.

3. Security That Starts at Generation

The first paper described a security gate that runs checks against generated code before approval. That is correct, but it is only half the picture. The gate is not where most of the security comes from. It comes from how the code is generated in the first place.

In a normal pipeline, a human writes a configuration, a scanner inspects it, a problem is found, and the change is fixed and re-checked. The value is real, but it arrives after the problem already exists. Under IAI, the agent reads the security rules as part of deciding what to generate. The rule that data storage must be encrypted, that access must be limited to known sources, that certain endpoints must not face the public internet — these are not tests the output has to survive later. They are conditions the output has to meet to be written at all. The agent produces an encrypted volume because the rules do not allow any other kind.

This changes what the gate is for. It is no longer the place where security is achieved. It is the place where it is confirmed. The agent generates a change that follows policy, and the gate independently checks that it does, using the engine's own analysis. Two separate safeguards remain — the agent's intent to follow the rules, and the gate's independent check — but the main work has moved to the start.

The table below sets out the difference:

	Detection model	IAI model
When security is applied	After the change is written	While the change is generated
What the scanner does	Finds problems to fix	Confirms the change already follows policy
What happens to a violation	Created, found, triaged, fixed	Never created
Where effort goes	Clearing a backlog of findings	Improving the policy rules

The practical result is that a whole category of work shrinks. The triage backlog exists because insecure changes are created all the time and have to be found. When changes are generated against policy, the number of new violations drops toward the number of gaps in the policy itself — a much smaller number, and one that keeps shrinking as the rules improve. The security team spends less time clearing findings and more time improving the rules.

4. One Rulebook, Always Enforced

If security comes from generation, then the rules that guide generation become the most important security asset the organization has.

In most organizations there are really three versions of the security policy. There is the standard written in a document. There is the smaller, slower set of checks that tooling actually enforces. And there is a third version: what the running infrastructure really looks like. The gaps between the three are where exposure hides — the rule everyone believes is in force, but nothing actually applies. Under IAI there is one version. The policy the agent enforces and the policy the organization declares are the same thing — security baselines and recognized control frameworks, written as code [5]. The agent reads that policy every time it generates a change and applies it every time. There is no hand translation from a written standard into scanner rules that quietly falls behind. What the organization has declared is what the infrastructure does.

Figure 3. Conventional security keeps three versions of the policy that drift apart, and the gaps between them are where exposure hides; under IAI the documented, enforced, and running policy are one rulebook, read on every change.

This also keeps the rules current. Because the agent maintains the policy alongside the Platform Manifest, the two move together. When a new type of resource appears, its policy is set as it is brought in, not discovered to be missing after an incident. When a standard changes, it applies to every new change at once, and the idle loop (Section 7) flags every existing resource that would now break it. And because the policy is code, it can be reviewed, versioned, and compared over time. The question

an auditor asks first — what was the rule, when did it take effect, and what changed it — is answered by looking, not by asking.

5. Load-Bearing Dependencies and Security

The first paper introduced load-bearing dependencies: criticality has to be declared, and it spreads through the dependency chain, so that anything a critical service depends on, inherits that importance. For security, this is what lets the system judge risk by consequence instead of treating every resource the same.

A scanner can tell you that an access rule is too open. It is much weaker at telling you what that open rule actually puts at risk. The seriousness of a misconfiguration is not just about the rule. It depends on what the resource can reach and what depends on it. A development sandbox and a production payment service should not be treated as equal. The scanner sees the rule. It does not see the dependency chain. The intent layer sees the chain, because the manifest holds it.

This changes three things in practice:

- **Severity is judged by what is at risk.** The same open access rule is not equally urgent everywhere. On an isolated sandbox it is minor. On a database that the payment service depends on, it is serious. Because the agent knows the dependency chain, it can rank what it shows the human by real consequence, instead of by a generic severity score that treats both the same.
- **The defaults get stronger where it matters.** When a resource is load-bearing — or inherits importance from something that depends on it — the agent generates it with tighter access, encryption switched on, and less exposure, without the operator having to ask. The most important parts of the infrastructure end up the most tightly controlled, automatically rather than by memory.
- **The whole chain is protected, not just the visible part.** A payment service is easy to spot and protect. The database, network path, identity provider, and storage it quietly depends on are easy to overlook. Because importance spreads through the dependency chain, IAI protects all of them — with segmentation between tiers and least-privilege paths between them — so the weakest link sitting behind a critical service is not left open [6].

6. The Supply Chain and Traceability

The recognition that the supply chain is now a main source of risk raises a direct question: what is actually being deployed, and do we know where it came from?

The intent layer governs provisioning, not application build. The infrastructure it provisions is increasingly immutable — built from a pre-made image produced by a separate process, and replaced rather than changed in place. This means the security of a running service depends heavily on the image it was built from. IAI can check that image because the configuration the agent generates already names it:

the agent reads that reference and scans the image — for known vulnerabilities and exposed secrets — as part of the same security gate that scans the configuration. Two policy rules keep the check reliable. First, the image must be named by a fixed digest rather than a moving tag, so the image that is scanned is exactly the image that is deployed. Second, no resource that deploys an image can pass the gate without a clean scan. Configuration and supply chain are checked together, on the same change, and the image and its scan result are recorded in the manifest.

This is where the manifest earns its place. Because it records, after every change, not only what exists and how it is set up but also what it was built from and why, it gives the infrastructure traceability: a running record of where everything came from and what relies on it [7]. When a new vulnerability is announced, the question "where, across everything we run, is the affected component, and what depends on it" is answered from the manifest instead of pieced together by hand under pressure.

The limit is worth stating plainly, the same way the first paper stated its limits. The intent layer does not build the images, does not own the application's internal security, and does not watch the running workload's behaviour. It governs what is provisioned, checks what is deployed, and records what was built. Where the supply chain meets the infrastructure — at the point of provisioning — IAI is the control. Past that point, responsibility sits elsewhere; IAI makes that shared-responsibility line explicit rather than blurring it.

7. Continuous Compliance and Drift as a Signal

When no request is pending, the same agent keeps working in the background, checking the infrastructure for drift and for breaks in policy. Continuous monitoring itself is not new — most mature organizations already have it. What is different is who is doing the monitoring, and what they can do about what they find.

A standalone monitoring tool watches the infrastructure from the outside. It can tell you a setting has changed, but it does not know what the setting was meant to be, or why the resource exists, so it raises an alert and waits for a person to work out the rest. The idle loop is run by the same agent that generates and approves changes. It already holds the declared state, the policy, and the reason each resource exists. So it does more than notice a change — it knows whether the change should have happened. And because it is the agent that makes changes, it can propose the fix, not just the alert.

This is also what turns drift into a security signal. The intent layer is the only approved way the infrastructure changes, and every approved change is recorded in the manifest. So a change that appears in the real infrastructure but not in the manifest did not come through the approved path. It is out-of-band — an unauthorized action, an external mistake, or a sign of compromise — and the loop

flags it, because the agent knows what the infrastructure should look like and this does not match. Nothing already running is trusted simply because it is there; what the agent did not do, it treats as suspect [6]. Put simply: if the agent did not make the change, it can see that the change was made. Detecting unapproved change is normally a hard problem. Here it falls out almost for free, and the same record gives the audit trail — who changed what, when, and why — as a by-product of normal work.

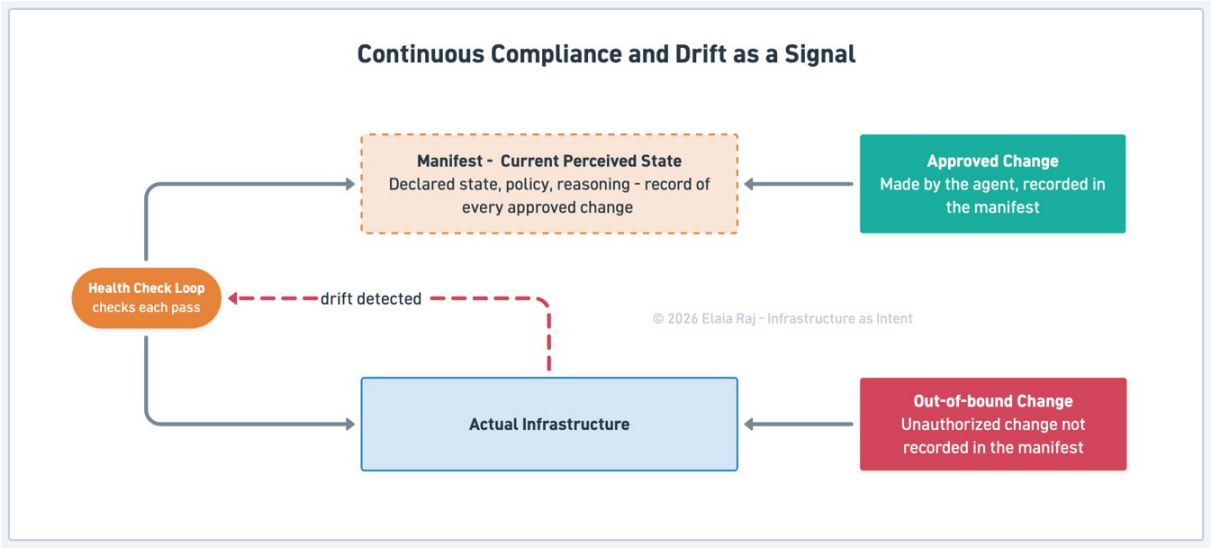

Figure 4. The health-check loop compares the actual infrastructure against the manifest on every pass, so an approved change is recognized while an out-of-band change that is not in the manifest is flagged as drift.

8. Autonomy Is a Security Decision

The first paper described earned trust as the way an agent comes to act on its own over time. Looked at from a security angle, the human approval step is itself a security control.

An agent that can create, change, and remove infrastructure is one of the most sensitive parts of the system. The first question to ask of it is not how much it can do, but how its power is limited. The same zero-trust posture an organization applies to its people should apply to the agent: no implicit trust, least privilege, and verify explicitly [6]. IAI does this in three ways:

- **Human approval.** For now, nothing is applied without a human decision against a plain-language summary. This is not a temporary inconvenience. It is the line that stops the agent's reasoning, however good, from turning straight into action with no one accountable. As Section 2 noted, this is also where the field has landed on its own.
- **Earned trust, by risk.** Freedom is given first for low-risk, repeated, well-understood changes, and held back — perhaps for good — for new changes to

business-critical or data-bearing systems. This is least privilege applied to the agent itself. The more a change matters, the more it has to prove before it runs without approval, and the most important changes keep human approval.

- **Identity, not stored keys.** The agent signs in with short-lived, identity-based credentials and holds no long-lived secret, and its identity is checked on every action rather than trusted once.

The field is right to be careful with agents that act on their own, and IAI builds the structure for it. This maps onto the maturity model from the first paper: autonomy is reached one level at a time, as the system proves itself.

9. Current Industry Position

Almost every piece of this already exists. Posture-management platforms rank findings well — but they work on infrastructure that is already written, and they detect rather than prevent. Policy-as-code engines enforce rules well — but they run as a gate inside a pipeline they do not own, so a path that skips the pipeline skips the policy. Security agents are arriving fast — but they are aimed at detection and response, watching the infrastructure from the outside rather than being the path it changes through. Supply-chain scanners check where components came from — but they sit to one side of provisioning. Each piece is strong on its own. None of them sits where every change has to pass.

That is the gap, and it is not a missing feature. It is the absence of a single intent layer that every change to the whole infrastructure must go through — one place that generates against policy so problems are never created, checks the supply chain before deployment, judges risk by the dependency chain, keeps watch through the idle loop, treats unapproved change as a signal, and limits its own power through approval and identity. The parts are on the shelf. Assembling them into one governed path is what IAI does, and what no single tool does today.

10. Conclusion

Every stage of infrastructure security has placed security after the work, because the work was something a human did and security could only follow. IAI changes who does the work, and so it changes where security can sit. When an agent generates infrastructure against policy, security moves ahead of creation. The misconfiguration is prevented instead of detected. Compliance is kept instead of checked. Risk is judged by what depends on what. The audit trail builds itself.

This is not a break from where security is heading. The field has decided that detection alone is not enough, that earlier must become smarter, that the supply chain matters as much as configuration, and that AI agents must act under human approval. IAI puts those decisions into one layer and applies them, at once, to

infrastructure that no single tool covers in full. Organizations that build this layer will not only find their problems faster. They will run infrastructure where a large class of problems cannot be created, cannot sit unnoticed, and cannot change without leaving a clear record of why.

References

[1] Datadog. *State of DevSecOps 2026*. Findings on the share of organizations running known, exploitable vulnerabilities in live cloud services. Reported in DeepStrike, "DevSecOps Statistics 2026." https://deepstrike.io/blog/devsecops-statistics

[2] Practical DevSecOps. "DevSecOps Trends 2026: AI, Supply Chain & Zero Trust." On the shift from shift-left to shift-smart, supply-chain hardening, and policy as code. https://www.practical-devsecops.com/devsecops-trends-2026/

[3] Sonatype. *2026 State of the Software Supply Chain*. On the scale of open-source supply-chain risk: more than 454,000 new malicious packages identified in 2025. https://www.sonatype.com/state-of-the-software-supply-chain/introduction

[4] Cloud Security Alliance. *The State of AI and Cybersecurity 2026* (survey of more than 1,500 security leaders). On how few practitioners will allow autonomous fixes with no human in the loop. https://cloudsecurityalliance.org/articles/the-state-of-ai-cybersecurity-2026-unveiling-insights-from-over-1-500-security-leaders

[5] Cloud Security Alliance. *Cloud Controls Matrix (CCM) v4.1*. A control framework of cloud security objectives, published in machine-readable form. https://cloudsecurityalliance.org/research/cloud-controls-matrix

[6] National Institute of Standards and Technology. *Special Publication 800-207, Zero Trust Architecture* (2020). On least privilege, segmentation, continuous verification, and the assume-breach posture. https://csrc.nist.gov/pubs/sp/800/207/final

[7] Cybersecurity and Infrastructure Security Agency. *2025 Minimum Elements for a Software Bill of Materials (SBOM)*. On recording software components for traceability and vulnerability response. https://www.cisa.gov/resources-tools/resources/2025-minimum-elements-software-bill-materials-sbom

---